



Trabalho de Programação Orientada a Objetos



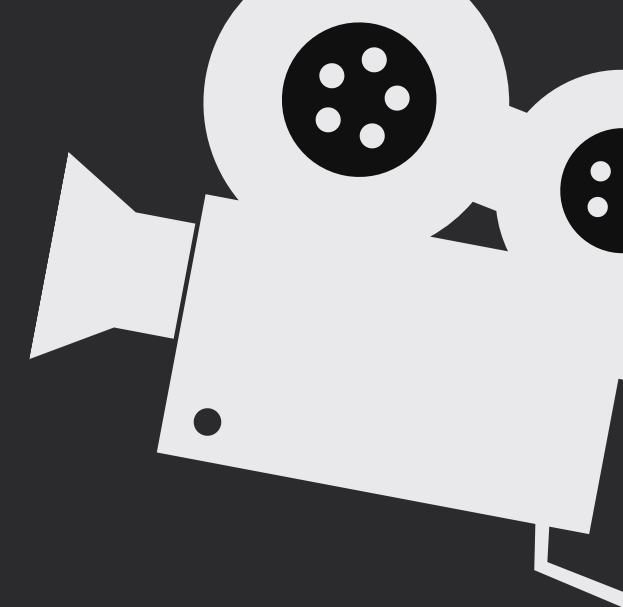
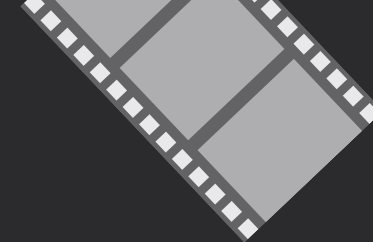
Catálogo de Filmes e Series

Professora: Roselene Henrique Pereira Costa

Faculdade: Pontifícia Universidade Católica

Data: 19/06/2026

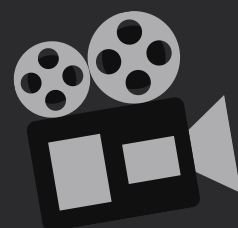




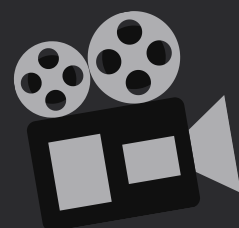
INTEGRANTES



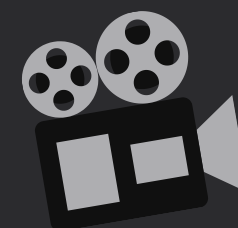
Eduardo
Niquini
Santiago



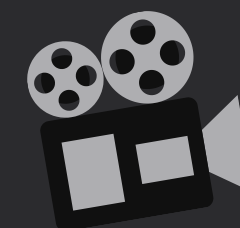
Fábio Sérgio
Carvalho
Mendonça



Guilherme
Cândido
Vidulino



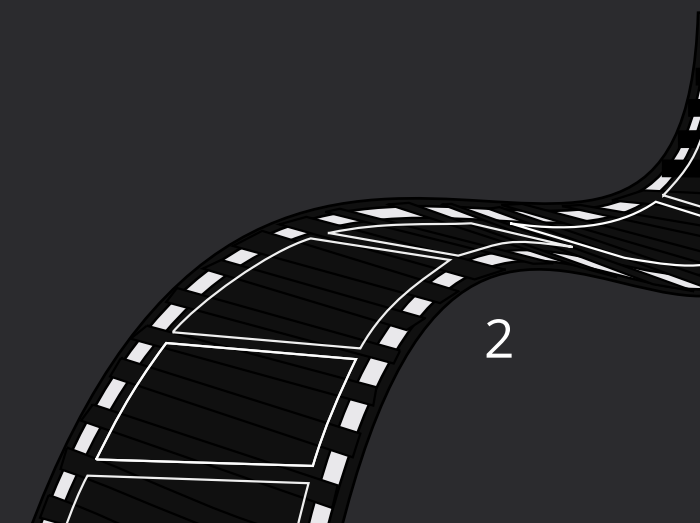
Heitor Freitas
Fernandes



Lucas Marinho
Blon Rocha



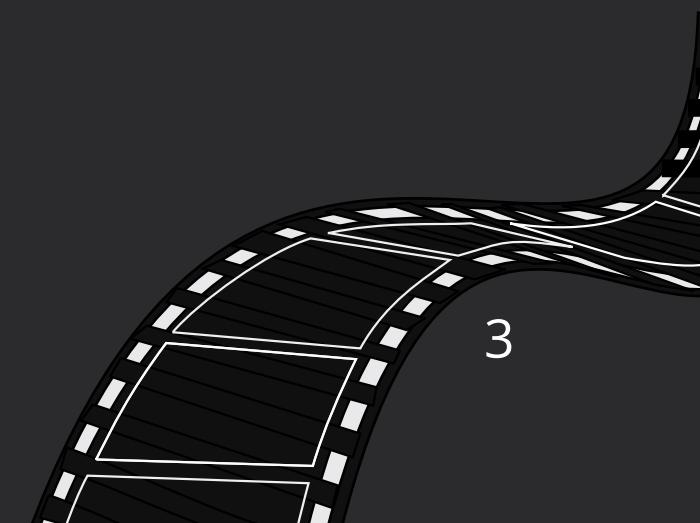
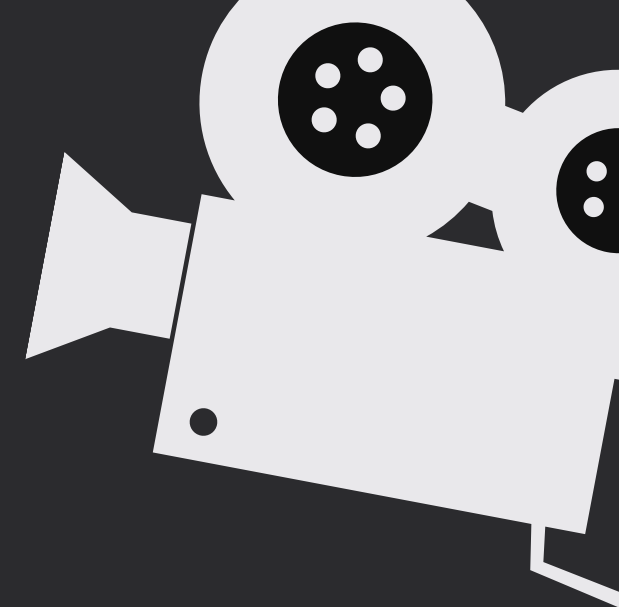
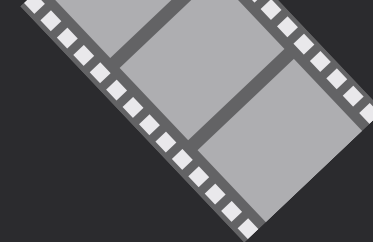
Matheus Souto
dos Santos





Sumário:

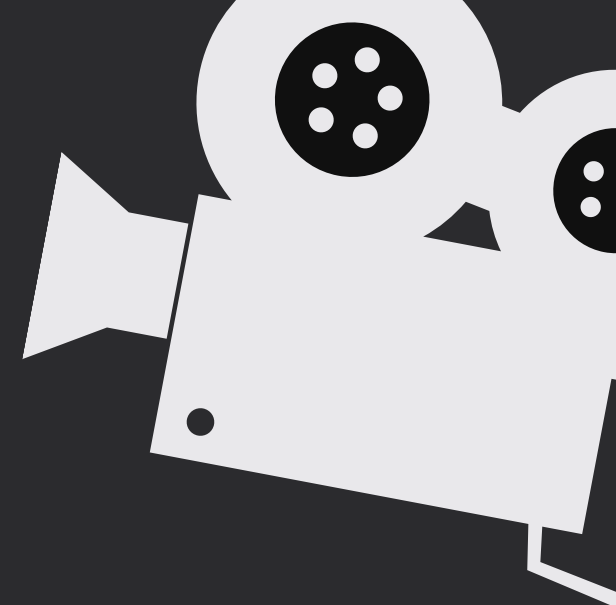
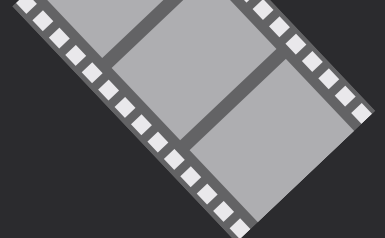
1.INTRODUÇÃO.....	4
1.1 Ideia do Projeto.....	5
2.IMPLEMENTAÇÕES.....	6
2.1. Estrutura do Projeto.....	7
2.2. Classe e Objetos.....	8
2.3.Heranças.....	9
2.4. Composição e Polimorfismo..	10
2.5. Interface e Encapsulamento ..	11
2.6. Construtores e Sobrescrita....	12
2.7.Tratamento de Exceção.....	13
3.Conclusão.....	14
3.1. Dúvidas.....	15
3.2 Finalização.....	16





INTRODUÇÃO

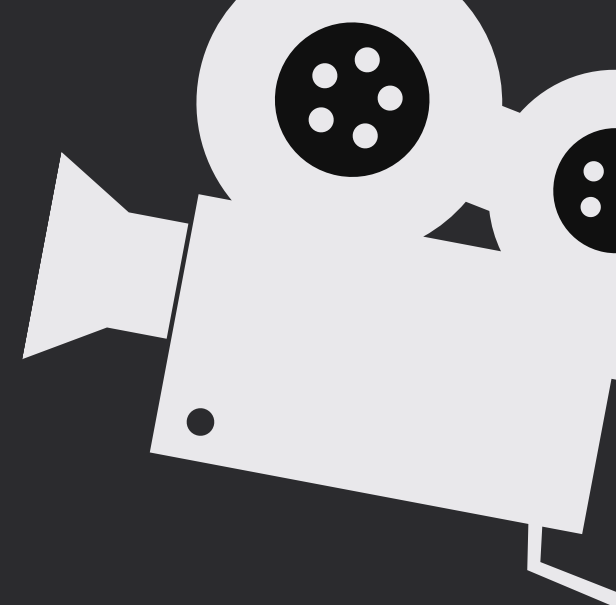
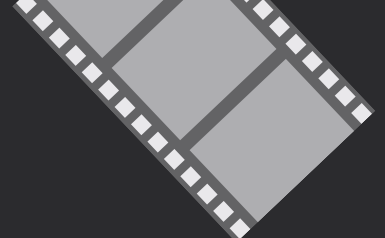
Demonstraremos e abordaremos os principais conceitos de Programação Orientada a Objetos, que foram adquiridos durante as aulas ao longo deste semestre e utilizados no desenvolvimento do sistema CineLog, mostrando como a abstração, a herança, o encapsulamento, o polimorfismo e as interfaces incentivam uma melhor organização, reutilização e manutenção do código.



IDEIA DO PROJETO

- 
1. Para que serve?
-O CineLog serve para organizar e gerenciar informações sobre filmes e séries, funcionando como um catálogo digital

- 
2. O que é o CineLog?
-CineLog é um software gerenciador de filmes e séries, construído em C#, que permite o registro de mídias, usuários, avaliações e listas personalizadas.



IMPLEMENTAÇÕES



Estrutura do Projeto:

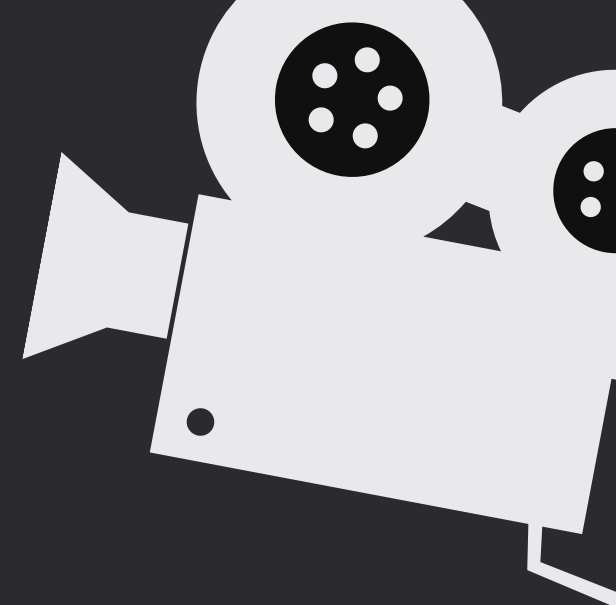
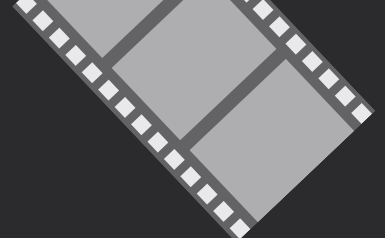
- Models: representam os objetos do sistema;
- Services: possuem as regras da aplicação;
- Repositories: controlam o armazenamento dos dados;
- UI: responsável pela interação com o usuário.



Classes e Objetos:

- Midia: classe geral;
- Filme: possui características próprias;
- Série: possui temporadas e episódios.

```
public class Filme
{
    public string Titulo { get; set; }
    public int Ano { get; set; }
}
```



ABSTRAÇÃO:



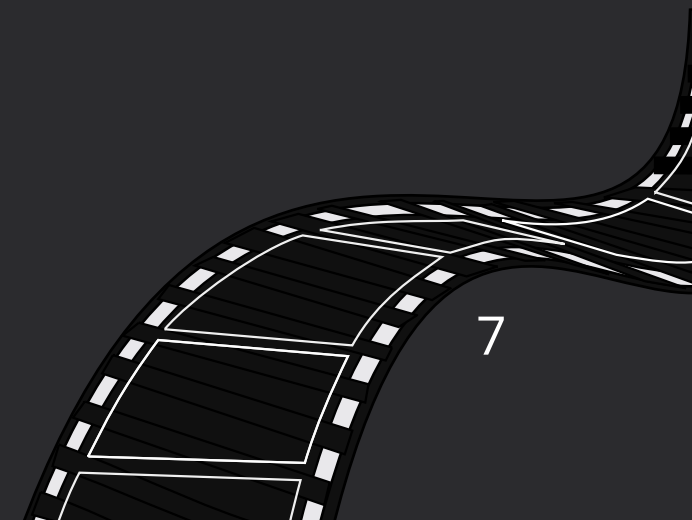
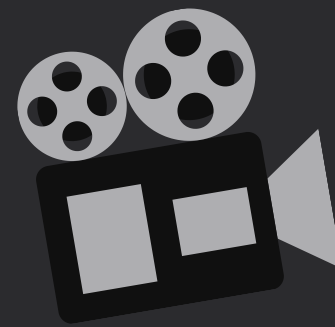
O que é:

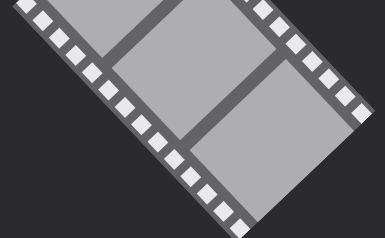
-A classe abstrata simboliza uma ideia geral. Não é possível criar uma instância dela diretamente.



Como foi usada:

```
public abstract class Midia
{
    // ENCAPSULAMENTO: propriedades com acesso controlado
    1 reference
    public int Id { get; protected set; }
```





HERANÇAS:

Herança Hierárquica



O que é:

-A herança permite à criação de vários filhos que possuem pelo menos uma característica de uma classe pai.

Como foi usada:

```
public class Filme : Midia  
{
```

```
public class Filme : Midia  
{
```

Herança Simples

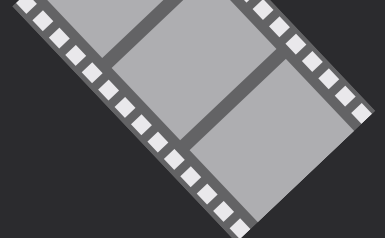
O que é:

-Uma classe recebe características de outra.

Como foi usada:



```
public class Serie : Midia  
{  
    2 references  
    public int NumeroTemporadas { get; private set; }  
    2 references  
    public int NumeroEpisodios { get; private set; }  
    2 references  
    public bool EmAndamento { get; private set; }  
    2 references  
    public string Criador { get; private set; }  
}
```



COMPOSIÇÃO E POLIMORFISMO

Composição



O que é:

-A composição permite que objetos sejam formados por outros objetos, como usuários possuindo avaliações e listas.

Como foi usada:

```
public class Usuario
{
    0 references
    public List<Avaliacao> Avaliacoes { get; set; }

    0 references
    public List<Midia> Favoritos { get; set; }
```

Polimorfismo

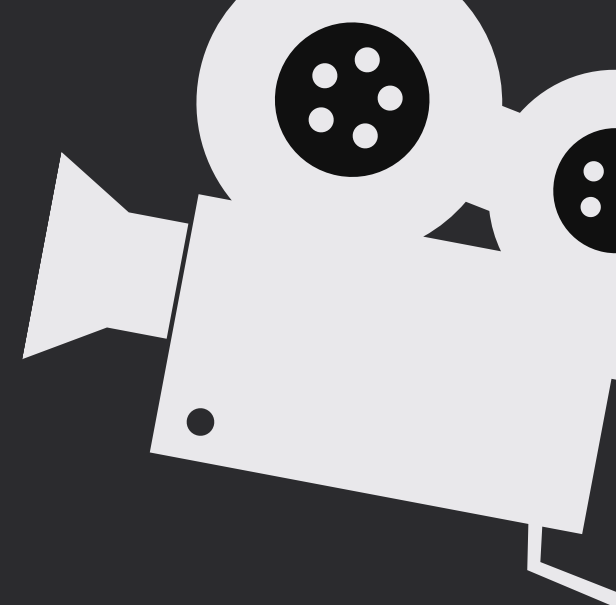
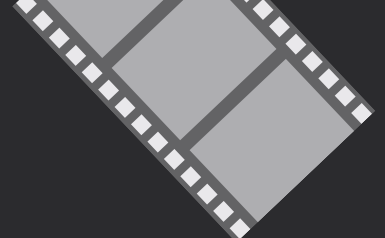


O que é:

-O polimorfismo permite que método apresente diferentes comportamentos de acordo com o objeto.

Como foi usada:

```
public abstract class Midia
{
    0 references
    public abstract string ObterTipo();
}
```



INTERFACE E ENCAPSULAMENTO

Interface

Escapsulamento



O que é:

-As interfaces definem regras que as classes devem seguir, deixando o sistema mais flexível.

Como foi usada:

```
public interface IMidiaRepositorio
{
    1 reference
    void Adicionar(Midia midia);

    0 references
    List<Midia> Buscar();
}
```

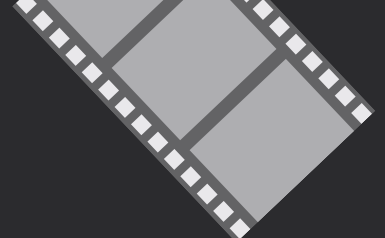


O que é:

-O sistema controla como informações podem ser alteradas, evitando mudanças incorretas e mantendo a segurança dos objetos.

Como foi usada:

```
public class Midia
{
    0 references
    public string Titulo { get; protected set; }
}
```



CONSTRUTORES E SOBRESCRITA

Construtores



O que é:

-Os construtores ajudam a criar objetos corretamente.

Como foi usada:

```
public Filme(int id, string titulo, int anoLancamento, string genero,  
             string sinopse, int duracaoMinutos, string diretor)  
    : base(id, titulo, anoLancamento, genero, sinopse)
```

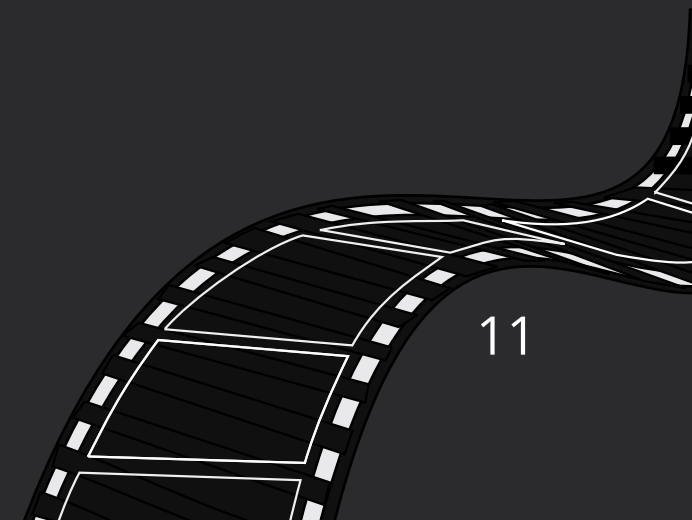
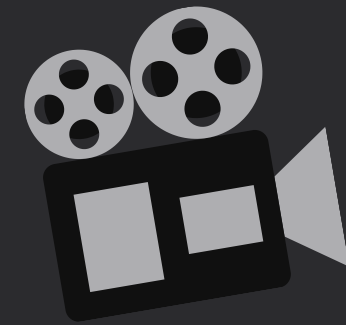
Sobrescrita

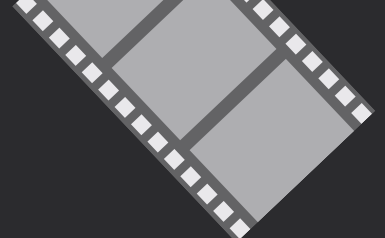
O que é:

-É quando uma classe filha reescreve um método que já existe na classe pai, criando um comportamento específico para ela.

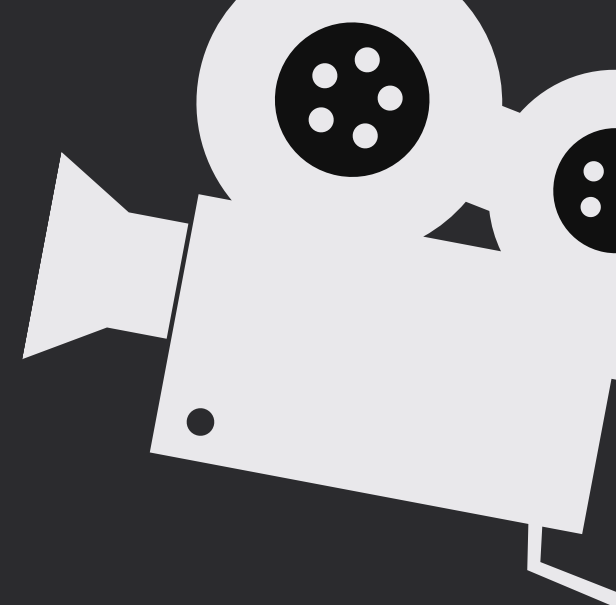
Como foi usada:

```
public override string ObterDetalhes()  
{  
    return $"Série: {Titulo}";  
}
```





TRATAMENTO DE EXCEÇÃO



O que é?

O tratamento de exceções permite controlar situações inesperadas durante a execução do programa.



Como foi Implementado/Utilizado?

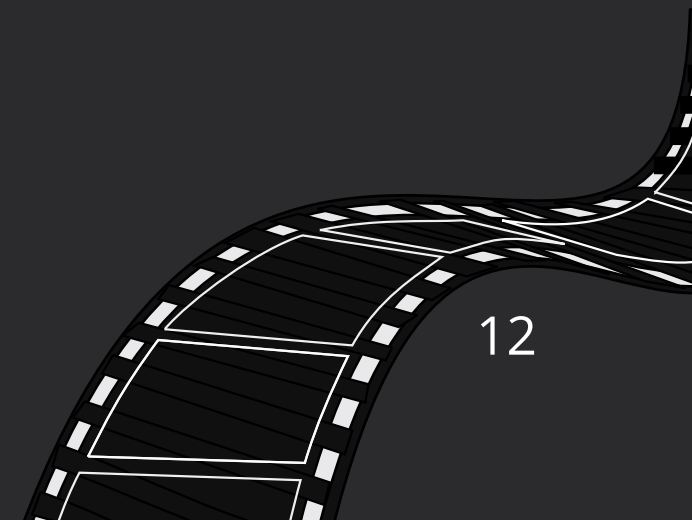
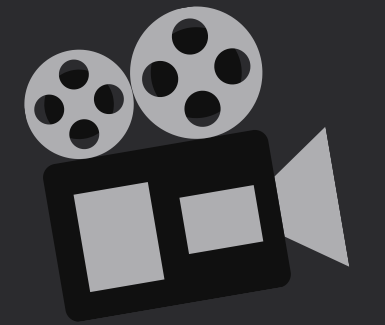
No CineLog, ele é usado para validar informações e informar ao usuário quando alguma operação não pode ser realizada.

Implementação:



```
public class CineLogException : Exception
{
    5 references
    public CineLogException(string message) : base(message) { }
}

1 reference
public class DadosInvalidosException : CineLogException
{
    0 references
    public DadosInvalidosException(string message) : base(message) { }
}
```

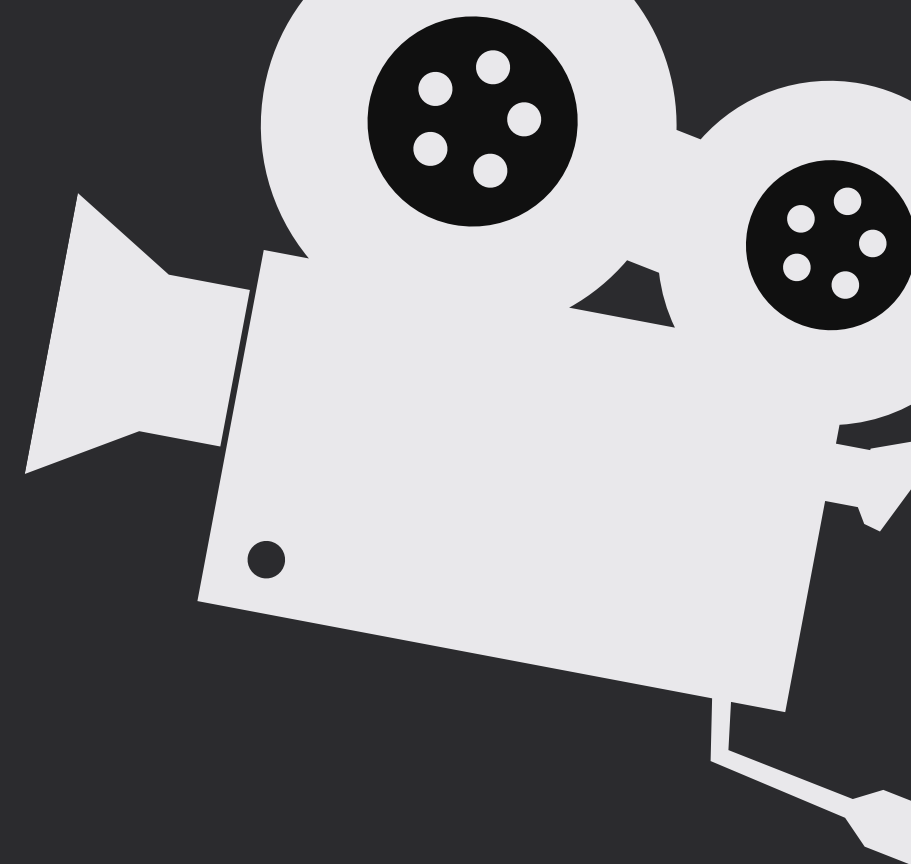
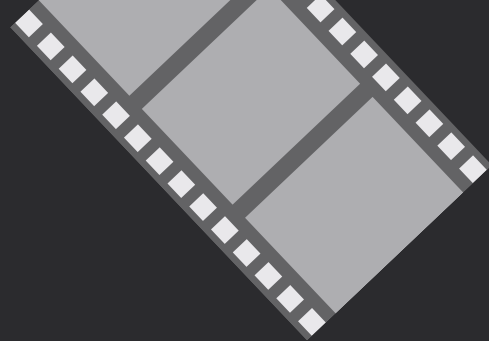




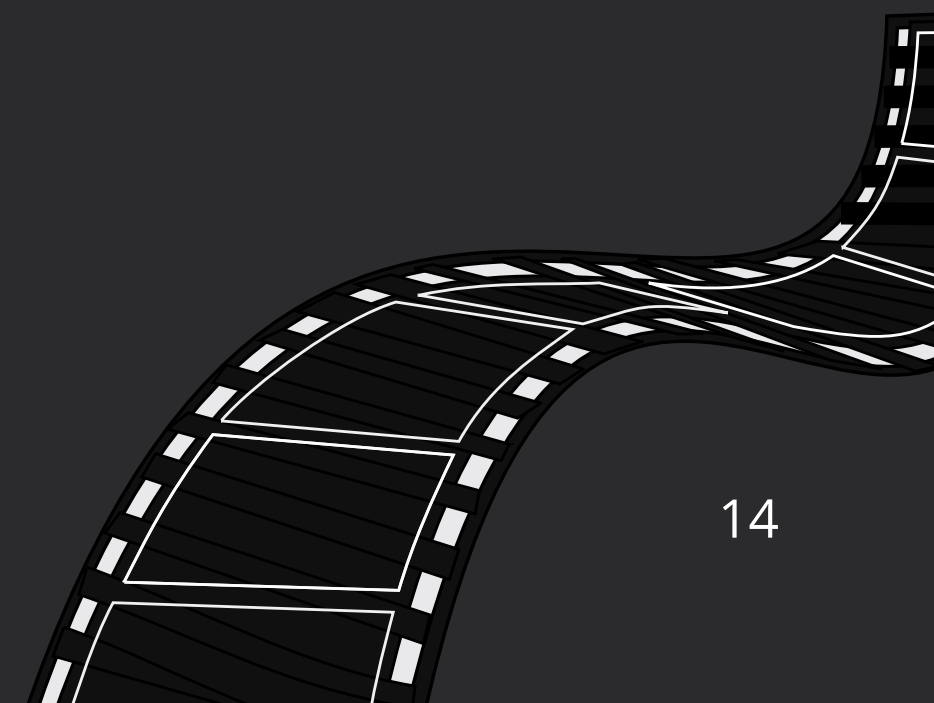
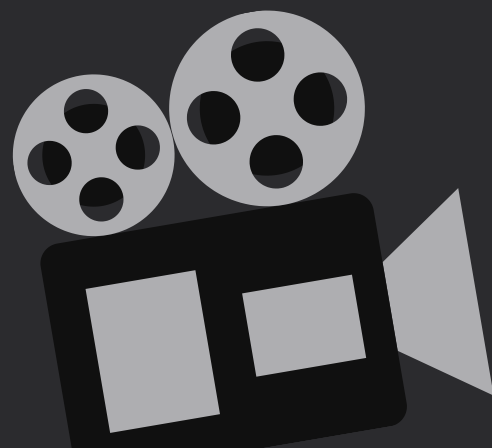
CONCLUSÃO

O projeto CineLog ilustra os conceitos mais relevantes da Programação Orientada a Objetos. Utilizando classes, herança, abstração, encapsulamento, polimorfismo e interfaces, o sistema consegue preservar um código que se mostra mais estruturado, reutilizável e de fácil manutenção.

Esses conceitos fazem a aplicação mais ajustável e preparada para melhorias no futuro.



ALGUMA DÚVIDA?



OBRIGADO
POR ASSISTIR

